

Coding - Conditional Statements

Sometimes we will only want to run code if certain requirements have been met. For example, we may want to move our player back to the start if their y-position falls below a certain threshold. To do this we use conditional statements. These statements use *conditions* to control when or how many times we run a block of code. The most common conditional that we'll see is the *if* statement.

if statements

An *if* statement is a conditional statement that runs a block of code when a condition is *true*. (Remember, the data type for *true* and *false* is called a boolean or bool). We will typically store this condition in a variable, or it will be a property of an object. This is a very powerful tool when programming. Below is a very basic example of an if statement:

```
if condition:
    #do something here when the condition is true
```

We can also pair an if statement with an *else* statement. An *else* statement runs when the *condition* is false. Here's an example:

```
if condition:
    #do something here if the condition is true
else:
    #do something different here when the condition is false
```

Comparison operators: checking for conditions

Often we will want to check one bit of data against another bit of data to see if it meets a condition. We can use the comparison operators to do this! These operators are placed between two pieces of data and when executed will return a boolean. Below is a list of the most common comparison operators:

Operator	Name	How it works
>	Greater than	Returns true if the left operand is a greater value than the right operand, otherwise returns false
<	Less than	Returns true if the left operand is a lesser value than the right operand, otherwise returns false

Operator	Name	How it works
>=	Greater than or equal to	Returns true if the left operand is of greater or equal value to the right operand, otherwise returns false
<=	Less than or equal to	Returns true if the left operand is of lesser or equal value to the right operand, otherwise returns false
==	Equals	Returns true if the left and right operands are the same value, otherwise returns false
!=	Does not equal	Returns true if the left and right operands are <i>different</i> values, otherwise returns false

Typically, we will use these operators with *ints* and *floats*. Below are some examples of what that would look like:

```
3 > 5 # This would return false
5 <= 5 # This would return true
28.6 == 28 # This would return false
0 != 12 # This would return true
```

However, we will never hard-code a comparison operator like above. Instead, we will typically use a variable or an object's property as one of the operators. Here's an example using a variable:

```
var my_number = 32

my_number < 80 # This would return true
my_number >= 32.01 # This would return false
my_number != 0 # This would return true
```

And here's an example using the position property of a `CharacterBody2D`:

```
@onready var my_character_body : CharacterBody2D = $MyCharacterBody

func _physics_process(delta: float) -> void:
    # This returns true if our player is left of the origin
    my_character_body.position.x <= 0

    # This returns true if our player is above the origin
    my_character_body.position.y < 0
```

Multiple Conditions: boolean operators

Some times we will want to check multiple conditions. We can do this through the use of boolean operators. These are special operators that use booleans (true and false) on each side, and return a boolean from them. Here are the most common operators:

Operator	Name	How it works
and	AND	Returns true if both operands are true, otherwise returns false
or	OR	Returns true if either operand is true, returns false if both operands are false

We will often use two comparison expressions on either side of a boolean operator to check if two conditions are being met. Here's some examples:

```
# false    true
32 > 0 and 16 != 12
# Because we are using an and operator and one of the expressions is false,
this returns false

# false    true
32 > 0 or 16 != 12
# Because we are using an or operator and one of the expressions is true, this
returns true

# true     true
32 != 0 and 16 != 0
# Because we are using an and operator and both expressions are true, this
returns true

# true     true
32 != 0 or 16 != 0
# Because we are using an or operator and both expressions are true, this
returns true
```

Putting it all together

Let's consider the following scenario: We have a game where our player will be teleported back to the origin if they move too high up or too far down. They will also be teleported back to the beginning if their health reaches 0 and then have their health reset to 100. Here's what our starting script may look like:

```
extends CharacterBody2D
```

```

const SPEED = 300.0
const JUMP_VELOCITY = -400.0
var health = 100

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

```

We would need to add conditional statements in the `physics_process()` function to check for these conditions every frame. Let's start with checking if the player is too low. For our purposes we'll say the threshold for "too low" is 2000. (Remember: the y axis is inverted in 2D)

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.

```

```

    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

    # Checks if the player is too low
    if position.y > 2000:
        position = Vector2(0,0)

```

Next let's add another condition to check if they're too high. Let's set our too high threshold to 0:

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

    # Checks if the player is too low
    if position.y > 2000:

```

```

        position = Vector2(0,0)

# Checks if the player is too high
if position.y < 0:
    position = Vector2(0,0)

```

You'll notice that both of these conditions run the same block of code when they are true. We can combine these conditions into a single if statement by using the *or* boolean operator

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

    # Checks if the player is too low or too high
    if position.y > 2000 or position.y < 0:
        position = Vector2(0,0)

```

Next let's handle checking the player's health and resetting their position and health:

```

extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)

    move_and_slide()

    # Checks if the player is too low or too high
    if position.y > 2000 or position.y < 0:
        position = Vector2(0,0)

    # Checks if the player is out of health
    if health <= 0:
        position = Vector2(0,0)
        health = 100

```

Notice that while each conditional code block shares the "position = Vector2(0,0)" line the health conditional has an extra line. This means we cannot combine it into a single if statement as we had done before.

Now it's your turn

In your Basket Catch project, in your player script, complete the following tasks:

1. Remove the conditional statement that is related to gravity

2. Remove the conditional statement that allows your player to jump
3. Add a conditional statement that doubles the velocity property if the following code is true.
This should be at the bottom of the physics_process() function *before* the move_and_slide() method:
 1. Input.is_action_pressed("ui_accept")
4. Add a conditional statement that prints the y position if the player's velocity on the x axis is not 0

Then answer the following in comments at the bottom of the script (below the physics_process() function):

5. Determine what would be returned by the following comparison operations:
 1. $28.5 > 28.5$
 2. $32 < 100$
 3. $0 \neq 2$
 4. $82 == 82.0$
 5. $79.5 \geq 78$
6. Determine what would be returned by the following boolean operations:
 1. $60 < 80$ or $10 > 0$
 2. $32 == 32$ and $78 < 100$
 3. $64 \neq 64$ and $0 < 100$
 4. $2 == 0$ or $3 > -6$
 5. $12 < 10$ or $100 > 0$

Note: After this assignment, your Basket Catch project should have a play area where the player cannot exit the sides or the bottom, three falling objects, your player should not be able to jump, and holding space should double the speed of your player.